

Swasthya AI

Insurance Sales Agent

Voice-first · Multilingual · Grounded · Agentic

20

Insurance products

67

Plan tiers

3

Languages
En · Hi · Hinglish

Why health insurance — not life?

 **< 40%**

Health insurance penetration

India is deeply underinsured on health. Life insurance penetration sits at 3.2% of GDP; health lags far behind despite medical costs inflating 10–12% annually.

 **₹5–35L**

A single critical illness bill

One hospital admission can wipe out a family's savings. Health cover fills the gap life insurance was never designed to address — it pays while you're alive.

 **20 products**

Personalisation opportunity

Health insurance has far more product surface area than life: age gates, PED types, maternity, chronic disease, gig workers, seniors — each warrants a distinct product.

Objective & Scope

What it does

- Inbound voice sales agent — answers the call, qualifies the lead, recommends a policy, and closes.
- Discovers 9 structured profile fields in a minimal number of turns.
- Recommends from a 20-product × 67-plan suite using algorithmic filtering.
- Grounds every number in a typed product registry — never invents a figure.
- Handles price objections with real treatment cost data.
- Sends WhatsApp confirmation on confirmed purchase via Twilio.

MVP scope decisions

Inbound only

No outbound dialling for MVP

3 languages

English · Hindi · Hinglish

Non-streaming STT→LLM→TTS

Full-turn architecture — simpler to debug, sufficient latency for demo context. Production would stream.

OpenAI Agents SDK

Replaces hand-rolled FSM as primary orchestrator for speed to demo

Sarvam STT + TTS

Saaras v3 + Bulbul v3 — Indic language advantage

FSM kept as fallback

If LLM unavailable, FSM takes over on same data

Architecture evolution: FSM → Agentic

Finite State Machine

S0 Start

S1 Discovery

S2 Recommend

S3 Policy Q&A

S4 Close · S5 Handoff · S6 Block

- ✓ Full control over transitions
- ✓ Every path is auditable + testable
- ✗ Managing all edge transitions gets complex
- ✗ Slower to iterate for MVP demo timeline

MVP

OpenAI Agents SDK

M_16 Sales Agent

Tool-calling loop drives the full conversation.
Model decides when to discover, recommend, or close.

7 grounded tools

save_profile

recommend_products

explain_product

show_plan_options

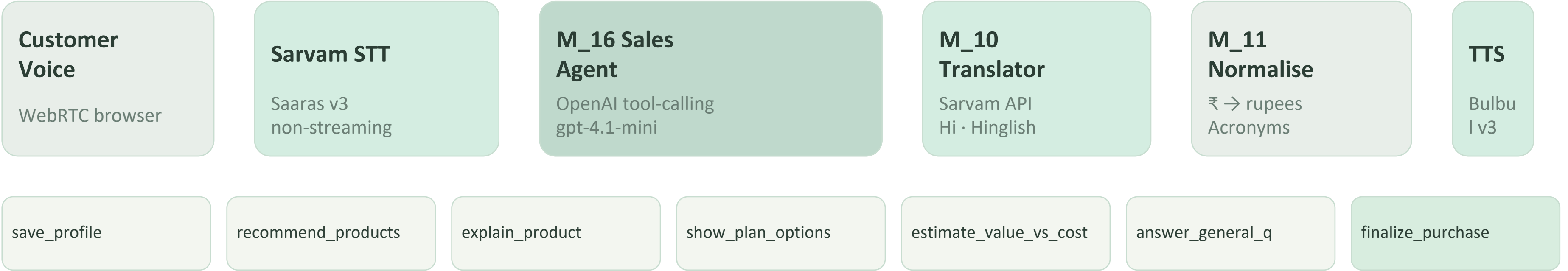
estimate_value_vs_cost

answer_general_question

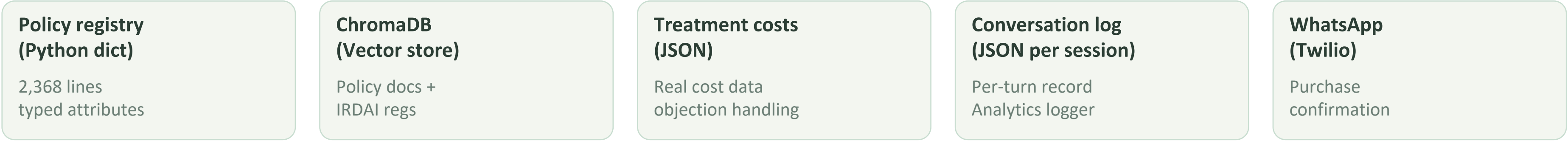
finalize_purchase

- ✓ Less boilerplate — faster to demo
- ✓ FSM kept as deterministic fallback
- ✓ Real OpenAI Traces for debugging

End-to-end pipeline



Data layer



Data & retrieval design

Deterministic lookup

Policy feature registry

policy_feature_registry.py

Single Python dict of dicts. 20 products × 75 typed attributes. Consulted by `recommend_products` and `show_plan_options`. Never injected into LLM context — served via tool call only.

JSON lookup

Treatment cost data

treatment_costs.json

Real hospitalisation cost ranges by procedure and city tier. Used by `estimate_value_vs_cost` to handle price objections with evidence rather than persuasion.

Semantic RAG

Vector store (ChromaDB)

2 indexes: policy docs + IRDAI regs

20 policy .txt files + 5 IRDAI regulatory documents chunked by section. Top-6 retrieval via text-embedding-3-small. Used by `explain_product` and `answer_general_question`.

Audit trail

Conversation log

logs/conversation_<session>.json

Every turn: user message, agent reply, intent, tool sequence, schema snapshot, LLM call trace. Enables session replay, analytics, and guardrail review.

What's deterministic, what's LLM

Deterministic (Python code)

Hard product filters

buyer_type, age, gender, primary_need — eliminate impossible products. Code, not inference.

Soft scoring

PED type, OPD need, budget, family size weighted and ranked. Returns top candidates + score.

Sufficiency gate

buyer_type + age + primary_need must all be non-null before retrieval runs.

Safety intent detection

UNSAFE / WANT_HUMAN / FRUSTRATED caught by keyword matching — never delegated to LLM.

NER + schema update

save_profile tool validates field values against enum sets before writing to schema.

Numeric guardrail (M_15)

Every number in reply re-checked against the tool output that was given to the model.

LLM-assisted (OpenAI gpt-4.1-mini)

Tool call decisions

Model decides which tool to call each turn — discover, recommend, explain, or close.

Response generation

Narrates tool results in natural spoken language. Constrained by system prompt + tool output.

Intent classification

LLM advisory layer on top of deterministic keyword check. Used for routing emphasis.

Question phrasing

Templates exist; LLM adapts wording to channel (voice vs WhatsApp) and conversation history.

RAG synthesis

Retrieves top-6 chunks, synthesises a grounded answer. M_15 checks numbers after.

Objection handling

Frames treatment-cost data into empathetic, evidence-based responses.

Discovery, NER & scoring logic

Question priority queue

- 1

buyer_type
Hard filter
- 2

age
Hard filter
- 3

gender
Hard filter
- 4

primary_need
Hard filter → often resolves alone
- 5

has_ped / ped_type
diabetes_cardiac → SP015
- 6

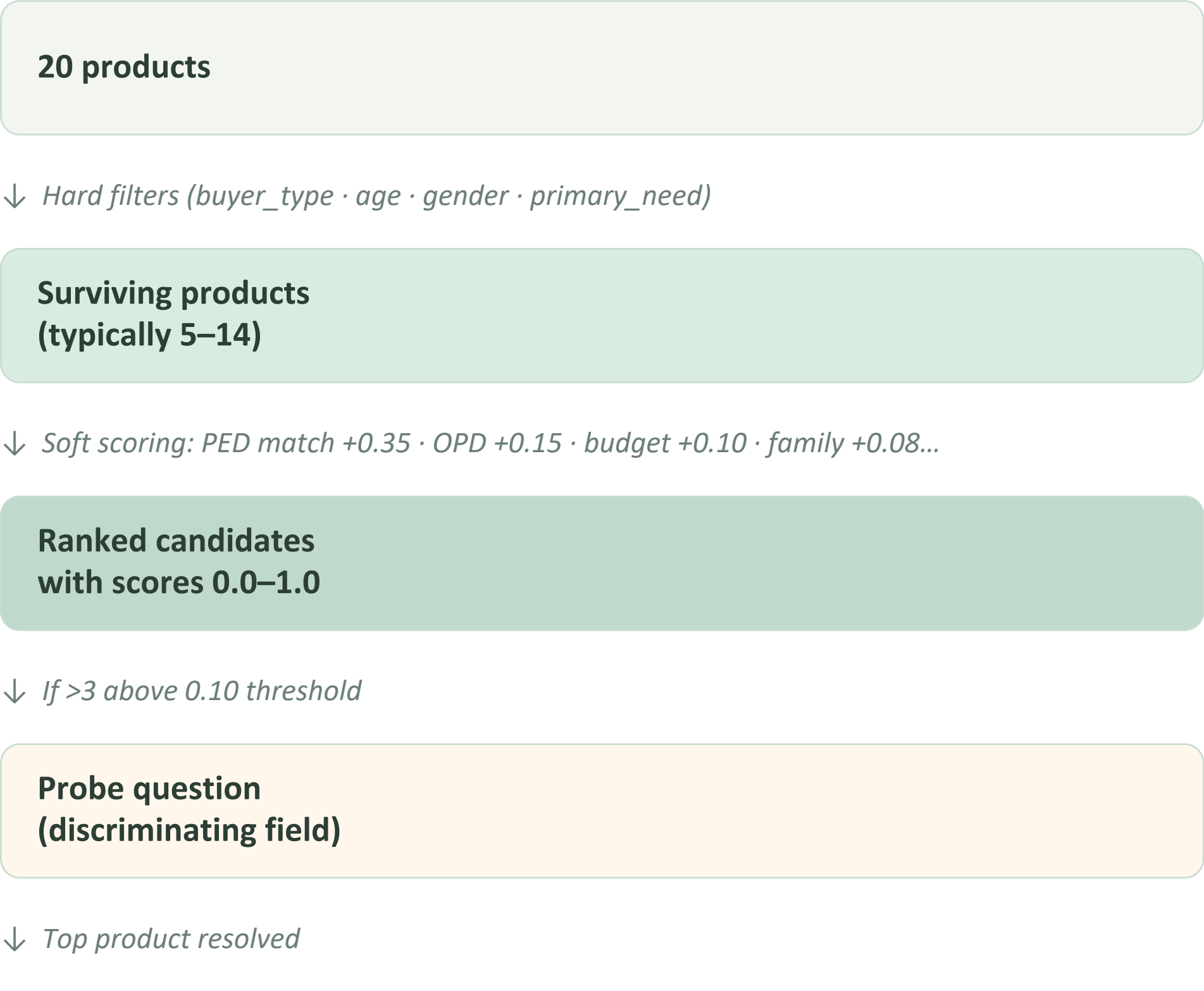
needs_opd
Eliminates 9 products if True
- 7

budget_band
Guardrail, not eliminator
- 8

family_cover + size
floater_joint → SP020
- 9

si_preference
Plan tier only — ask after product resolved
- Minimum for retrieval: fields 1 + 2 + 4

Retrieval scoring



Guardrails, closure & WhatsApp

Advisory

M_15 Numeric Guardrail

Every premium, waiting period, and benefit figure in the reply is re-checked against the tool output the model was given. Flags ungrounded numbers. Does not block the response — flags for review.

Always-on

M_11 Response Normaliser

₹ → 'rupees', 300000 → '3,00,000', IRDAI → 'I R D A I'. Strips markdown. Caps at 2,500 chars on a sentence boundary. Ensures TTS receives clean, speakable text.

Hard block

Safety exits

UNSAFE (prompt injection) → S6 Blocked.
WANT_HUMAN / FRUSTRATED → S5 Handoff. Both caught by deterministic keyword matching before the LLM sees the turn.

Purchase closure flow

User says
'yes'

finalize_
purchase tool

Record sale
+ session log

M_14 WhatsApp
Agent (Twilio)

Customer gets
confirmation

Tech stack & key takeaways

LLM

OpenAI gpt-4.1-mini
OpenAI Agents SDK 0.17.5

Voice STT / TTS

Sarvam Saaras v3 (STT)
Sarvam Bulbul v3 (TTS)

Voice framework

Pipecat 1.3.0
Silero VAD + FastAPI

RAG / Vectors

LlamaIndex + ChromaDB
text-embedding-3-small

Messaging

Twilio WhatsApp sandbox

Translation

Sarvam Translate API

Key lessons

- One source of truth — product registry as ground truth eliminates hallucination at the source.
- Guardrails cost less than validation LLMs — M_15 + M_11 catch most errors cheaply.
- Deterministic fallback — FSM stays live so the agent never goes dark if the LLM is slow.
- Multilingual from day one — Sarvam STT/TTS doubles addressable market with minimal extra code.
- Voice latency is UX — every 100ms saved between STT and TTS is perceptible to the customer.